

```

#include <stdio.h>           // Provides printf(), fprintf(), fgets(), perror(), stdin and
stderr
#include <sys/types.h>       // Defines system data types used in socket programming
#include <sys/socket.h>      // Provides socket(), connect(), send() and socket constants
#include <netinet/in.h>      // Provides sockaddr_in, AF_INET and htons()
#include <netdb.h>           // Provides gethostbyname() and struct hostent

#define SERVER_PORT 5432    // Port number on which the server is running
#define MAX_LINE 256        // Maximum number of characters that can be read at a time

int main(int argc, char *argv[]) // Main function receives command-line arguments
{
    struct hostent *hp;         // Stores information about the server host
    struct sockaddr_in sin;     // Stores the server's IP address and port number
    char *host;                // Stores the server hostname
    char buf[MAX_LINE];        // Buffer used to store text entered by the user
    int s;                     // Socket descriptor
    int len;                    // Number of bytes to be sent

    /*
     * Check whether the user has supplied exactly one argument:
     * the hostname or IP address of the server.
     */
    if (argc == 2)
    {
        host = argv[1];        // Store the server hostname given on the command line
    }
    else
    {
        // Display the correct format for running the client program
        fprintf(stderr, "Usage: simplex-talk host\n");

        exit(1);               // Terminate the program because the argument is missing
    }

    /*
     * Convert the server hostname into its corresponding IP address.
     */
    hp = gethostbyname(host);   // Obtain host information using the hostname

    if (hp == NULL)             // Check whether hostname resolution failed
    {
        // Display an error message when the hostname cannot be found
        fprintf(stderr,
            "simplex-talk: unknown host: %s\n",
            host);

        exit(1);               // Terminate the program
    }

    /*
     * Build and initialize the server-address structure.
     */
    bzero((char *)&sin, sizeof(sin)); // Set all bytes of the address structure to zero

    sin.sin_family = AF_INET;     // Specify that an IPv4 address is being used

    /*
     * Copy the server IP address obtained from gethostbyname()
     * into the sockaddr_in structure.
     */
    bcopy(
        hp->h_addr,              // Source: resolved server IP address
        (char *)&sin.sin_addr,   // Destination: IP-address field of sin
        hp->h_length              // Number of address bytes to copy
    );

    /*
     * Store the server port number.
     * htons() converts the port from host byte order
     * to network byte order.

```

```

    */
    sin.sin_port = htons(SERVER_PORT);

    /*
     * Create a TCP socket.
     */
    s = socket(
        PF_INET,                // IPv4 protocol family
        SOCK_STREAM,            // Stream socket, which means TCP
        0                        // Use the default protocol for TCP
    );

    if (s < 0)                  // Check whether socket creation failed
    {
        perror("simplex-talk: socket"); // Display the system error message

        exit(1);                // Terminate the program
    }

    /*
     * Connect the client socket to the server.
     */
    if (connect(
        s,                      // Client socket descriptor
        (struct sockaddr *)&sin, // Server address
        sizeof(sin)              // Size of the server-address structure
    ) < 0)
    {
        perror("simplex-talk: connect"); // Display the connection error

        close(s);                  // Close the socket before terminating

        exit(1);                  // Terminate the program
    }

    /*
     * Main communication loop:
     * read lines from the keyboard and send them to the server.
     */
    while (fgets(buf, sizeof(buf), stdin) != NULL)
    {
        /*
         * Ensure that the last buffer position contains
         * the string-termination character.
         */
        buf[MAX_LINE - 1] = '\0';

        /*
         * Calculate the length of the string.
         * The additional 1 includes the terminating '\0' character.
         */
        len = strlen(buf) + 1;

        /*
         * Send the entered text to the server.
         */
        send(
            s,                    // Connected socket descriptor
            buf,                  // Address of the text buffer
            len,                  // Number of bytes to send
            0                     // No special sending options
        );

    }

    close(s);                    // Close the socket when input finishes

    return 0;                    // Indicate successful program completion
}

```

```

/*
 * Simple TCP Server Program
 *
 * This program creates a TCP server that:
 * 1. Creates a socket
 * 2. Assigns a port number
 * 3. Waits for client connection
 * 4. Receives messages from client
 * 5. Displays received messages
 */

#include <stdio.h>           // Provides printf(), fprintf(), perror(), stdout
#include <sys/types.h>        // Defines system data types used in socket programming
#include <sys/socket.h>       // Provides socket(), bind(), listen(), accept(), recv()
#include <netinet/in.h>       // Provides sockaddr_in structure and Internet address constants
#include <netdb.h>            // Provides network database functions

#define SERVER_PORT 5432     // Port number on which server will listen
#define MAX_PENDING 5       // Maximum number of pending client connections
#define MAX_LINE 256        // Maximum size of message buffer

int main()
{
    struct sockaddr_in sin;   // Structure to store server IP address and port details

    char buf[MAX_LINE];       // Buffer to store received message from client

    int len;                  // Stores length of received data

    int s;                   // Server socket descriptor

    int new_s;               // New socket descriptor for connected client

    /*
     * Build address data structure
     */

    bzero((char *)&sin, sizeof(sin));
    // Initialize the socket address structure with zeros

    sin.sin_family = AF_INET;
    // Specify IPv4 address family

    sin.sin_addr.s_addr = INADDR_ANY;
    // Accept connections from any available network interface

    sin.sin_port = htons(SERVER_PORT);
    // Assign port number 5432 to the server
    // htons() converts host byte order to network byte order

    /*
     * Create passive TCP socket
     */

    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0)
    {
        // Create TCP socket
        // PF_INET : IPv4 protocol family
    }

```

```

// SOCK_STREAM : TCP socket
// 0 : Default TCP protocol

perror("simplex-talk: socket");
// Display socket creation error

exit(1);
// Terminate program if socket creation fails
}

/*
 * Bind socket to server address and port
 */
if ((bind(s, (struct sockaddr *)&sin, sizeof(sin))) < 0)
{
    // Associate socket with IP address and port number

    perror("simplex-talk: bind");
    // Display binding error

    exit(1);
    // Terminate program if binding fails
}

/*
 * Put server socket into listening mode
 */
listen(s, MAX_PENDING);
// Server waits for incoming client connection requests
// MAX_PENDING = maximum number of waiting clients

/*
 * Main server loop
 * Wait for client connection,
 * receive data and display it
 */
while (1)
{
    len = sizeof(sin);
    // Store size of client address structure

    /*
     * Accept a client connection
     */
    if ((new_s = accept(s,
        (struct sockaddr *)&sin,
        &len)) < 0)
    {
        // accept() creates a new socket for communication
        // with the connected client
    }
}

```

```

        perror("simplex-talk: accept");
        // Display accept error

        exit(1);
        // Terminate program
    }

    /*
     * Receive data from client
     */

    while (len = recv(new_s, buf, sizeof(buf), 0))
    {

        // recv() receives data sent by client
        //
        // new_s : connected client socket
        // buf   : buffer to store received data
        // sizeof(buf): maximum bytes received
        // 0      : no special options

        fputs(buf, stdout);
        // Display received message on server screen

    }

    /*
     * Close client connection
     */

    close(new_s);
    // Close the client socket after communication ends

}

}

```